

BetterYou

Presentation Guide

CSE 4456 - iOS Application Development

MIT Manipal | Spring 2026

Surya Prakash Seth (230907028)

Naman Ahuja (230961174)

Gamified iOS Fitness Tracking App

SwiftUI | SwiftData | Swift Charts | SF Symbols

30 Swift Files | 2,100+ Lines of Code | 5-Tab Interface

STEP 1: Open Xcode & Show Project Structure

(~2 minutes)

Action: Open BetterYou.xcodeproj in Xcode. Expand the file navigator on the left sidebar.

Say:

"This is BetterYou -- a gamified fitness tracking app. The idea is to turn workouts into a game with XP, levels, quests, and an in-app store. Let me walk you through the project structure first."

Point to each folder and explain:

- **BetterYouApp.swift** -- "This is our app entry point. It sets up the SwiftData model container for persistence and forces dark mode."
- **Theme/** -- "AppTheme.swift holds all our color constants and fonts. ViewModifiers.swift has custom modifiers like .neonGlow() and .surfaceCard() so we don't repeat styling code."
- **Models/** -- "Our 5 data models using SwiftData -- UserProfile, Workout, Quest, StoreItem, WeightLog, plus Enums.swift for all our type definitions."
- **Services/** -- "Business logic separated from views -- XPEngine handles XP/coin awards and leveling, QuestEngine handles quest refresh and progress tracking, SeedData bootstraps initial data on first launch."
- **Screens/** -- "One folder per tab -- Home, Quests, Stats, Store, Profile, plus Root for the TabView shell."
- **Components/** -- "Reusable UI components -- PrimaryButton, ProgressBar, GlowingBadge, LevelUpSheet."

If teacher asks how many files/lines:

"30 Swift files, about 2,100 lines of code total, all using Apple native frameworks with zero third-party dependencies."

STEP 2: Show the Storyboard

(~3 minutes)

Action: Click Main.storyboard in the left navigator.

Say:

"Here is our storyboard showing the full app architecture. We designed this first to plan all our screens before writing the SwiftUI code."

Point out the layout:

- "At the center is a **Tab Bar Controller** with 5 tabs -- Home, Quests, Stats, Store, Profile."
- "Each tab connects to a **Navigation Controller** via a relationship segue, which then connects to the actual **View Controller**."
- "The Quests screen has a **modal segue** -- see this arrow -- that presents the Log Workout Sheet when the user taps the button."
- "Every screen has the dark theme applied -- background color #0B0B12, with green accent #39FF88 for buttons and XP elements."

Point to specific UI elements on each screen:

- **Home:** Level number label, XP progress bar, Lifetime XP and Streak info cards, quest previews, Log Workout button
- **Quests:** Daily/Weekly segmented control, quest cards with progress bars and reward labels, floating action button
- **Log Workout:** Type picker (Strength/Cardio/Bodyweight/Custom), text fields for name, duration, reps, weight, notes, Save button
- **Stats:** Three KPI labels (Workouts, Time, Volume), weekly progress bar, body weight chart area
- **Store:** Coin balance display, avatar items with Equipped/Buy states, Streak Freeze power-up
- **Profile:** Avatar display, title badge, editable name field, body weight, About section with team names, Reset button

If teacher asks why Navigation Controllers:

"Navigation Controllers give us the navigation bar at the top with the screen title, and they support push navigation if we want to add detail screens later."

STEP 3: Show the Data Models

(~3 minutes)

Action: Open Models/UserProfile.swift in the navigator.

Say:

"We have 5 data models using SwiftData. SwiftData is Apples modern replacement for Core Data -- it uses the @Model macro instead of .xcdatamodeld files."

Walk through UserProfile.swift:

- "UserProfile is a singleton -- only one instance exists. It stores the players name, avatarID, selectedTitle, currentXP, lifetimeXP, level, coinBalance, streak data, and bodyweight."
- "The xpToReachLevel() static method calculates the threshold -- Level N requires N times 500 XP."
- "We have computed properties xpInCurrentLevel and xpProgress that calculate the progress ring percentage on the Home screen."

Open Models/Workout.swift:

- "Each workout stores its type as an enum (strength, cardio, bodyweight, custom), plus name, duration, notes, and sets."
- "The sets are stored as JSON-encoded Data because SwiftData doesnt directly support arrays of custom structs. We encode and decode using Codable."
- "Theres a computed totalVolume property that calculates reps multiplied by weight across all sets."

Open Models/Enums.swift:

- "We defined 4 enums -- WorkoutType, QuestCadence, TargetMetric, StoreCategory -- all conforming to String and Codable so SwiftData can persist them."
- "The SetEntry struct is Codable and Hashable so it can be serialized into the Workout model as JSON data."

STEP 4: Explain the Business Logic

(~3 minutes)

Action: Open Services/XPEngine.swift.

Say:

"We separated all game logic from the views into dedicated service files. Views only handle UI -- all business logic lives here."

Walk through XPEngine.swift:

- "awardWorkout() -- adds 50 XP and 10 coins whenever a workout is saved, then calls recalculateLevel()."
- "awardQuest() -- adds the quests XP and coin reward when a quest is completed."
- "recalculateLevel() -- uses a while loop to check if lifetimeXP has crossed the next level threshold. Level N needs N times 500 total XP."
- "purchaseItem() -- checks coin balance with a guard statement, deducts coins, handles the streak freeze cap of 3. Returns a boolean for success or failure."

Open Services/QuestEngine.swift:

- "refreshIfNeeded() -- runs on app launch. Checks if daily quests were assigned before today's start-of-day. If so, deletes old ones and picks 3 new random templates. Same logic for weekly quests, checking against Monday."
- "recomputeProgress() -- after every workout save, loops through all active quests, filters workouts by the relevant time window (today for daily, this week for weekly), and recalculates progress based on the target metric."
- "When a quest completes, it awards XP and coins exactly once using the isCompleted flag to prevent double-counting."

Open Services/SeedData.swift:

- "12 quest templates -- 6 daily and 6 weekly -- with different target metrics like workout count, duration, volume, and distance."
- "Store catalog with 8 avatars using SF Symbols, 8 badges with titles, and 1 consumable Streak Freeze."
- "bootstrapIfNeeded() runs on first launch to create the UserProfile singleton and seed all store items into the database."

STEP 5: Run the App -- Live Demo

(~8 minutes -- this is the main part)

Action: Press Cmd+R to run on the iPhone simulator.

5a. HOME SCREEN

"Home screen shows the level badge with an animated XP progress ring. Below that are two info pills showing Lifetime XP and Streak count. Then a preview of today's quests with progress bars, and the primary Log Workout button at the bottom."

Point out: the green glow effect on the level number, purple progress bars on quest previews, the green Log Workout CTA button.

5b. QUESTS SCREEN

Action: Tap the Quests tab at the bottom.

"Quests are auto-generated -- 3 daily and 3 weekly. The segmented control at top switches between them. Each quest card shows the title, description, a progress bar, and the XP plus coin reward."

Action: Tap Weekly segment to show weekly quests.

"These refresh automatically -- daily quests reset at midnight, weekly quests reset on Monday."

5c. LOG A WORKOUT (Key Demo)

Action: Tap the + Log Workout floating button.

"This is the workout creation sheet. The type picker at top changes the form dynamically based on the workout type."

Demo a strength workout step by step:

- Leave type as Strength
- Type workout name: "Bench Press"
- Type duration: "30" (minutes)
- Show the sets section -- "We can add multiple sets with reps and weight"
- Set reps to 10, weight to 40 kg
- Tap the + button to add Set 2, enter reps 8 and weight 50
- Tap Save

"When we save, three things happen simultaneously: XP Engine awards 50 XP and 10 coins, Quest Engine rechecks all active quest progress, and if we cross a level threshold, a celebration sheet with sparkle animation appears."

Action: Briefly switch to Cardio type (don't save, just show the form change).

"See how the form changes dynamically -- for Cardio it shows a distance field instead of sets, reps, and weight."

5d. STATS SCREEN

Action: Tap the Stats tab.

"Stats shows three KPI cards at top -- total workouts, total training time, and total volume lifted. The weekly progress bar shows workouts done this week out of a goal of 4."

Demo the weight chart:

- "This chart uses Swift Charts -- Apples charting framework -- to draw a line graph of body weight entries."
- Tap the + button next to Body Weight
- Enter 72.5 and tap Save
- "See -- the chart updates live with the new data point."

5e. STORE SCREEN

Action: Tap the Store tab.

"The store shows the coin balance at top. Items are organized in sections -- Avatars, Badges and Titles, and Power-ups."

- "Runner avatar shows Equipped because its the default free avatar."
- "Other items show their coin cost and a Buy button."
- "The Buy button is visually disabled and greyed out when you dont have enough coins -- this uses .disabled() modifier in SwiftUI."
- If you have enough coins, buy an item: "Watch -- it gives haptic feedback using UIImpactFeedbackGenerator on successful purchase."

5f. PROFILE SCREEN

Action: Tap the Profile tab.

"Profile shows the equipped avatar and title. The avatar picker grid only shows avatars you own. The title picker only shows badges you have purchased."

Demo:

- Tap Edit on the name, change it to something, tap Save
- Point to About section: "Version 1.0.0, team names, course info"
- Point to Reset Progress: "This is a developer utility that clears all data for testing purposes"

STEP 6: Explain the UI Design

(~2 minutes)

Action: Open Theme/AppTheme.swift.

"All colors and fonts are centralized in this one file. We never hardcode hex values directly in any view."

Element	Color	Hex Code
Background	Near-black	#0B0B12
Surface cards	Dark grey	#16161F
Primary accent (XP, buttons)	Electric green	#39FF88
Secondary accent (Quests)	Violet	#9D6BFF
Tertiary (Streaks, coins)	Amber	#FFD24A
Primary text	White	#FFFFFF
Secondary text	Light grey	#B5B5C8

Action: Open Theme/ViewModifiers.swift.

- ".neonGlow() adds a colored shadow for the gaming glow effect"
- ".surfaceCard() adds padding, dark background, 16pt corner radius, and a subtle border -- every card in the app uses this modifier"

About SF Symbols (Icons):

"All icons in the app come from SF Symbols -- Apples built-in icon library with over 5,000 symbols. We reference them by name using Image(systemName:). For example: figure.run for running, bolt.fill for XP, flame.fill for streaks, snowflake for streak freeze, target for quests, bag.fill for store. These icons scale automatically and match the system font weight."

STEP 7: SwiftUI Concepts Used

(Reference -- mention if teacher asks)

SwiftUI Feature	Where Used	What It Does
@main	BetterYouApp.swift	App entry point
.modelContainer()	BetterYouApp.swift	Sets up SwiftData persistence
@Query	HomeView, StatsView, etc.	Fetches data, auto-refreshes UI
@Environment	LogWorkoutSheet, StoreView	Access to insert/delete data
@State / @Binding	Throughout all views	Local state + parent-child data flow
TabView + .tag()	RootTabView	5-tab navigation (= UITabBarController)
NavigationStack	Every screen	Nav bars (= UINavigationController)
.sheet()	QuestsView	Modal presentation (= present modally)
Picker .segmented	QuestsView, LogWorkoutSheet	Segmented controls
TextField	LogWorkoutSheet, ProfileView	Text input fields
ForEach	Quest lists, Store items	Dynamic lists (= UITableView)
Chart + LineMark	WeightChartView	Swift Charts line graph
.preferredColorScheme	BetterYouApp	Forces dark mode app-wide

STEP 8: Teacher Questions & Answers

(Prepare these -- your teacher will likely ask some of them)

Q: Why SwiftUI instead of UIKit/Storyboards?

"We learned the fundamentals of iOS development -- navigation controllers, tab bars, segues, MVC pattern -- in class using UIKit. For the project, we applied those same architectural concepts using SwiftUI, which is Apples recommended framework since 2019. The storyboard we designed shows we understand the UIKit approach too."

Q: How does data persist when you close the app?

"SwiftData handles persistence automatically. When we decorate a class with @Model, SwiftData creates a SQLite database behind the scenes. Any changes to model objects are saved automatically when the context is committed. This is similar to Core Data but with a much simpler API using Swift macros."

Q: How do you prevent buying items with insufficient coins?

"In XPEngine.purchaseItem(), the very first line is a guard statement that checks coinBalance >= item.cost. If the user doesnt have enough coins, the function returns false and nothing happens. In the UI, the Buy button is also visually disabled using the .disabled() modifier so the user can see they cant afford it."

Q: How does the level system work?

"Level N requires N times 500 lifetime XP. So Level 2 needs 1000 cumulative XP, Level 3 needs 1500, and so on. When a workout or quest awards XP, recalculateLevel() uses a while loop to check if lifetimeXP has crossed the next threshold. If it did, we show a celebration sheet. Every 5th level also grants a bonus streak freeze."

Q: How do quests refresh automatically?

"On every app launch, QuestEngine.refreshIfNeeded() checks the assignedAt date of existing quests. If daily quests were assigned before todays start-of-day, we delete them and randomly pick 3 new templates from our pool of 6. Weekly quests check against this Monday. This ensures fresh quests every day and week."

Q: What design pattern did you use?

"We used a service-oriented architecture. All views only handle UI rendering -- the business logic lives in XPEngine and QuestEngine service structs. This is similar to MVC where the Controller responsibility is split into dedicated service files. Views observe data using @Query and call service methods on user actions like saving a workout."

Q: How did you make the dark gaming theme?

"All colors are defined in AppTheme.swift using hex values converted to SwiftUI Color via a custom Color extension initializer. The .neonGlow() modifier adds a colored .shadow() for the glow effect. We force dark mode at the root level with .preferredColorScheme(.dark) so the system light mode setting cannot override our theme."

Q: Where do all the icons come from?

"Every icon in the app comes from Apples SF Symbols -- a built-in library of over 5,000 system icons. We reference them by name like figure.run, bolt.fill, flame.fill using Image(systemName:). No external image assets were imported. SF Symbols automatically scale with Dynamic Type and match the system font weight."

Q: How does the weight chart work?

"We use Swift Charts, Apples charting framework introduced in iOS 16. WeightChartView uses @Query to fetch the last 12 WeightLog entries from SwiftData, then renders them as a LineMark chart with PointMark for visible data points. The chart auto-updates whenever a new weight entry is saved because @Query observes the database."

Q: Can data survive app restarts?

"Yes -- we can demonstrate this live. Log a workout, then force-quit the app in the simulator by swiping up, then relaunch it. All the data including XP, level, workouts, and quests will still be there because SwiftData persists everything to a SQLite file on disk automatically."

STEP 9: Closing Statement

Say:

"To summarize -- BetterYou is a fully functional gamified fitness tracking app with 5 tabs, local data persistence using SwiftData, a dynamic quest system that refreshes daily and weekly, an XP and leveling engine, an in-app store with a separate coin economy, and live statistics with Swift Charts. We built 30 Swift files with about 2,100 lines of code, all using Apples native frameworks with zero third-party dependencies. Thank you."

Pre-Presentation Checklist

- Install the iOS Simulator runtime (Xcode > Settings > Platforms > download iOS 18)
- Do a clean build -- Cmd+Shift+K then Cmd+B -- make sure it compiles
- Reset app data before demo (Profile > Reset Progress, or delete app from simulator)
- Log 2-3 test workouts so Stats and Quests have real data to show
- Log 3-4 weight entries at different dates so the chart has visible points
- Make sure you have enough coins to demo a Store purchase (~20+ coins)
- Test the full flow once: launch > log workout > check quests > check stats > buy item > check profile
- Keep Xcode open with Main.storyboard visible as the starting point

Good luck tomorrow!